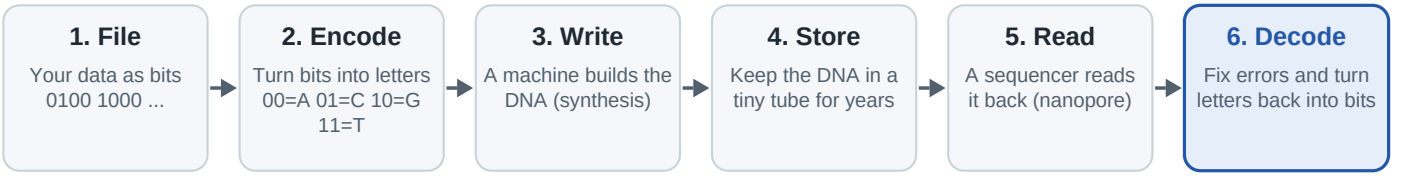
Storing Data in DNA

How it works, and why reading it back is hard

1 The idea: use DNA as a hard drive

A computer stores everything as **0s and 1s**. DNA stores information with **4 letters: A, C, G, T**. If we translate 0s and 1s into DNA letters, we can store files inside DNA molecules. DNA is extremely small, lasts for hundreds of years, and needs no electricity to keep the data.



Our project works here

Example: the word "Hi"

H i → `01 00 10 00  01 10 10 01` → C A G A C G G C

Each pair of bits becomes one letter. In our project every DNA strand is 128 letters long.

2 The problem: reading DNA back makes mistakes

The sequencer does not read perfectly. It makes three kinds of mistakes:

Original	A C G T A C G T C A	The correct strand.
Substitution	A C G **A** A C G T C A	One letter is **changed**. Everything else is still in the right place. **Easy** to fix.
Insertion	A C G T **T** A C G T C A	An **extra** letter appears. Every letter after it moves one place right.
Deletion	A C G ␣ A C G T C A	A letter is **missing**. Everything after it moves left, and we **do not know where** the gap is. **Hardest** to fix.

Why one deletion does so much damage

Original	A C G T A C G T C A	Compare letter by letter: after the missing letter, **almost every position looks wrong**, even though only one letter was lost. The more deletions, the worse this gets. This is the problem our project focuses on.
After deletion	A C G A C G T C A	
	ok ok ok x x x x x x x	

The help we have: the sequencer reads each strand **many times**. In our project we use **10 noisy copies** of every strand. Each copy has different mistakes, so by comparing all 10 we can work out the original.

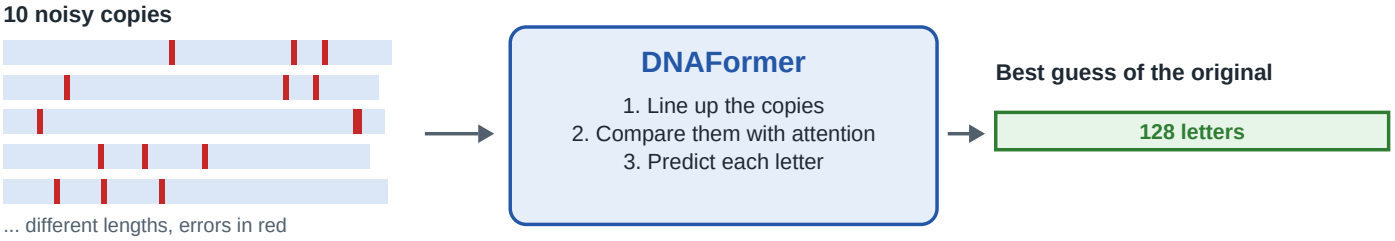
Key point: deletions are the hardest error because letters shift and we cannot see where the gap is.

DNAFormer and Our Idea

An AI that cleans up noisy reads, and how we try to help it

3 DNAFormer: an AI that rebuilds the original strand

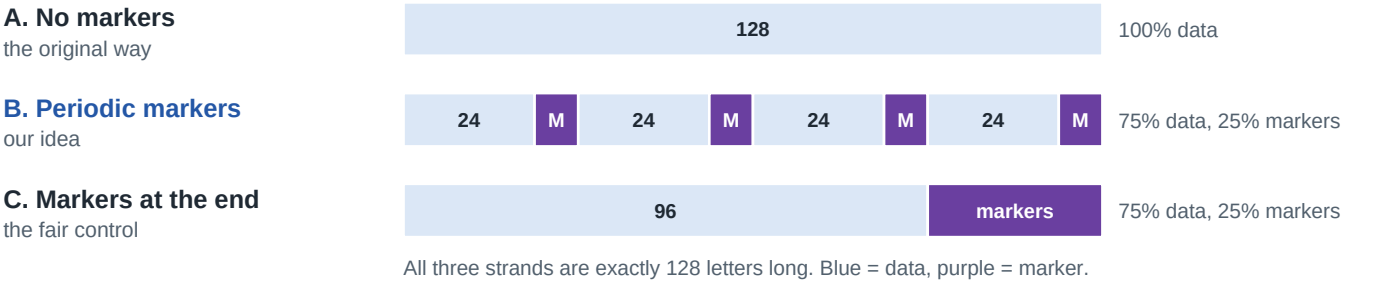
DNAFormer is a deep learning model (a **Transformer**, the same family as ChatGPT) published in *Nature Machine Intelligence* (2025). It looks at all the noisy copies of a strand together and predicts the original 128 letters. We use the **already-trained** model.



Its weak spot: DNAFormer was trained on reads with about **1% deletions**. At 5 to 8% deletions the copies drift out of line and it struggles.

4 Our idea: add signposts inside the DNA (synchronization markers)

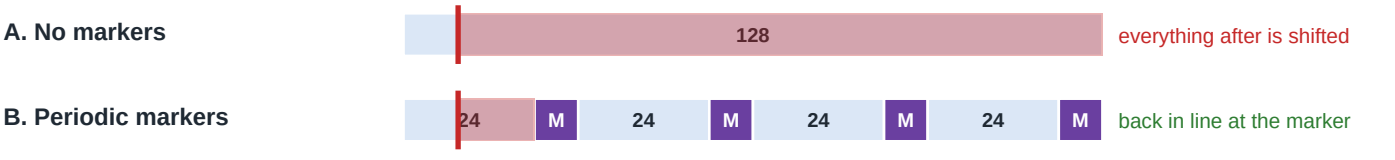
Think of **kilometre stones on a highway**. If you lose count of where you are, the next stone tells you exactly. We do the same: every 24 letters we place a fixed 8-letter signpost **`ACGTCAGT`**, called a **marker**. The model can use it to get back in line after a deletion.



All three strands are exactly 128 letters long. Blue = data, purple = marker.

Why layout C? B and C carry the **same amount of data** and the **same marker letters**. The only difference is the **spacing**. So if B does better than C, we know the gain comes from the markers being spread out, not just from having less data to recover.

What we hope happens after one deletion



Red line = deletion. Red shading = letters out of place.

What we keep the same

- DNAFormer's AI design (no redesign)
- Strand length: 128 letters
- 10 copies per strand, same error settings

What we change

- **Fine-tune** (extra training) DNAFormer on each layout
- Stop it throwing away short reads (at 8% deletion it would drop 92% of them)

We never tell the model where the markers are. It has to learn to use them during fine-tuning.

How We Test It

A fair experiment, clear measurements, and an honest decision rule

5 The experiment, step by step

Make data
Same data, same 10 copies, same errors for A, B and C

→

Baselines
Simple majority vote and DNAFormer without extra training

→

Fine-tune
Train DNAFormer on A, B and C with the same training budget

→

Test
On new strands at 4 deletion levels

→

Compare
Remove markers, score only the data letters

DONE and checked

Four test conditions (deletion rate)

Insertions and substitutions stay at 1%.

1% Light	3% Medium	5% Heavy	8% Severe

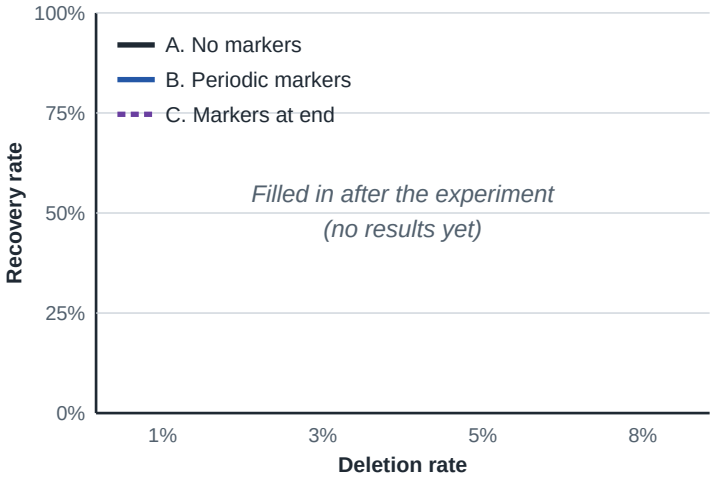
Data per layout: 10,000 training strands | 1,000 checking strands | 1,000 test strands for each deletion level

6 What we measure

Measure	What it means in simple words	Good is
Recovery rate	Share of strands rebuilt with every letter correct	Higher
Edit distance	How many letters are wrong, missing or extra (per letter)	Lower
Per-letter accuracy	Share of individual letters that are correct	Higher
File recovery	Does the whole file come back perfectly?	Yes
Information rate	How much of the strand is real data (A: 100%, B and C: 75%)	Higher
Runtime, GPU memory	How much time and computer power it needs	Lower

Marker letters are removed before scoring. Only the real data letters count, so B and C get no free points.

7 The final graph and how we decide



A second graph shows edit distance the same way.

If: B beats both A and C, mostly at 5 to 8%
Then: Markers help, and it is because of their spacing.

If: B is about the same as C
Then: The gain came only from having less data to recover, not from the markers.

If: B is not clearly better than A
Then: Markers are not worth losing 25% of the storage space.

Our research question in one sentence: With the same strand length, the same number of reads and the same errors, do evenly spaced markers help DNAFormer recover data better when there are many deletions?

We only claim markers help if the experiment shows it clearly and consistently.